LLM-Based Domain Model and Class Diagram Generation

Problem Statement

An Airline Ticketing System allows the Clerk to order tickets and to look for a suitable flight. In order to look up a ticket, the Clerk needs to enter the name of the traveler, the chosen flight, and finally booking the airline tickets. When looking up a flight, system provides a list of available flights between two airports at a given date. There is also an option of multicity, where the customer chooses to fly through multiple destinations with halting. The system must allow the ability to look up a flight without having to order a ticket. When the order information to buy a ticket is entered, this information is automatically used to look up the flight. A third service (the help service) is available for the clerk while ordering a ticket to provide a help page specific to ordering tickets. The help service can be invoked independently by the Clerk, which will provide the Clerk with a “main” help page.

Develop a LLM-based pipeline for deriving class diagram for the given problem. In this problem you will write a domain model, i.e., a conceptual class diagram (or analysis domain model). It should include attributes, methods with visibility specifiers and relationships that are useful for the application logic layer. You may assume certain characteristics for your classes, providing justification for any assumptions made regarding state. However, do not include conceptual classes for external systems or basic classes that would typically be found in the programming language, such as strings. You can include classes that are more useful versions of classes you would expect to find in the programming language.

Objectives and Requirements

1. **Domain Model Development**

   Develop a **LLM-based pipeline** for deriving a **class diagram** for the given problem.

   The generated domain model should:

   - Represent a conceptual class diagram (analysis domain model).
   - Include attributes, methods with visibility specifiers, and relationships that are useful for the application logic layer.
   - Allow assumptions regarding class characteristics, provided appropriate justification is given for any assumptions made regarding state.
   - Exclude conceptual classes for external systems or basic classes that would typically be found in the programming language, such as strings.

- Include classes that are more useful versions of classes you would expect to find in the programming language.

2. **Class Identification**

   Identify all classes required for the given Airline Ticketing System.

3. **Attribute Identification**

   Identify the attributes for each class.

4. **Method Identification**

   Identify the methods for each class.

5. **Visibility Specification**

   Identify the appropriate visibility specifiers for the identified methods and attributes.

6. **Relationship Identification**

   Identify the relationships among the classes using appropriate naming conventions.